PHISHING EMAIL DETECTOR

Project introduction, working procedure and screenshots

1. Introduction

The **Phishing Email Detector** is a web-based application that uses machine learning and natural-language text processing to classify an email as either **legitimate** or **phishing / suspicious**. Phishing emails often try to trick people into revealing passwords, banking information or other private details by using misleading links, urgent requests or false warnings. This project provides a quick first check of an email's text before a user decides how to handle it.

2. Technologies used

• **Python:** programming language used for model training and application logic.
• **Machine learning:** classifies email text based on patterns learned from labelled examples.
• **Text vectorization:** converts email words into numerical features that the model can process.
• **Flask:** connects the prediction code to the web application.
• **HTML and CSS:** provide the user interface displayed in the browser.
• **Render:** hosts the deployed web application.

3. How the project works

Step 1 — Prepare the data: labelled email examples are stored in CSV files. The labels identify examples as legitimate or phishing.
Step 2 — Train the model: run **python train_model.py**. The script processes the email text, creates numerical text features, trains the classifier, evaluates it and saves the trained model and vectorizer as PKL files.
Step 3 — Start the web application: Flask loads the saved model and vectorizer and serves the page from **app.py** using the HTML template in the **templates** folder.
Step 4 — Enter an email: the user pastes the email content into the text box and selects **Analyze Email**.
Step 5 — Predict and display: the entered text is transformed using the saved vectorizer and passed to the trained model. The page displays the predicted category and a confidence score. The Clear button removes the entered text.

4. Project files at a glance

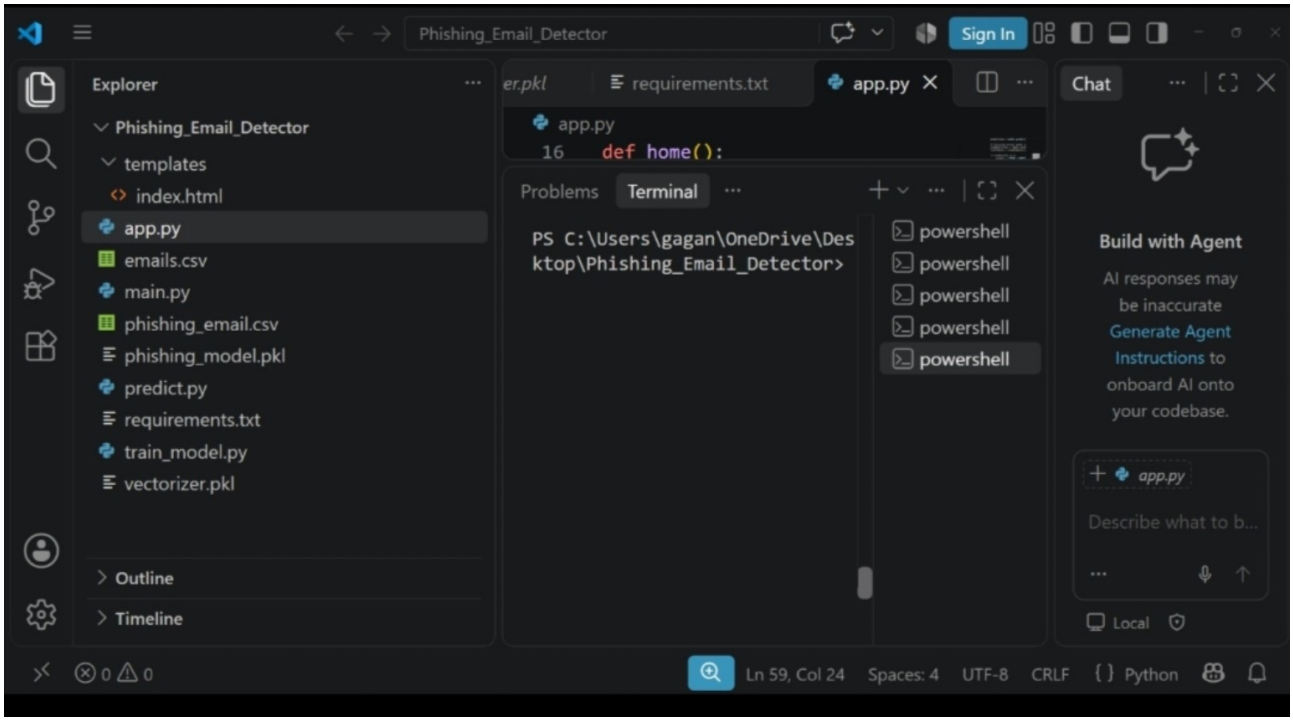
app.py — Flask routes and prediction request handling.
templates/index.html — browser interface.
train_model.py — trains and evaluates the model.
predict.py / main.py — supporting prediction or project logic.
emails.csv / phishing_email.csv — email datasets.
phishing_model.pkl / vectorizer.pkl — saved model and text vectorizer.
requirements.txt — Python packages required by the application.

5. Screenshots in project workflow order

The following screenshots are arranged from the source-code setup and model training through the deployed interface and its two example predictions.

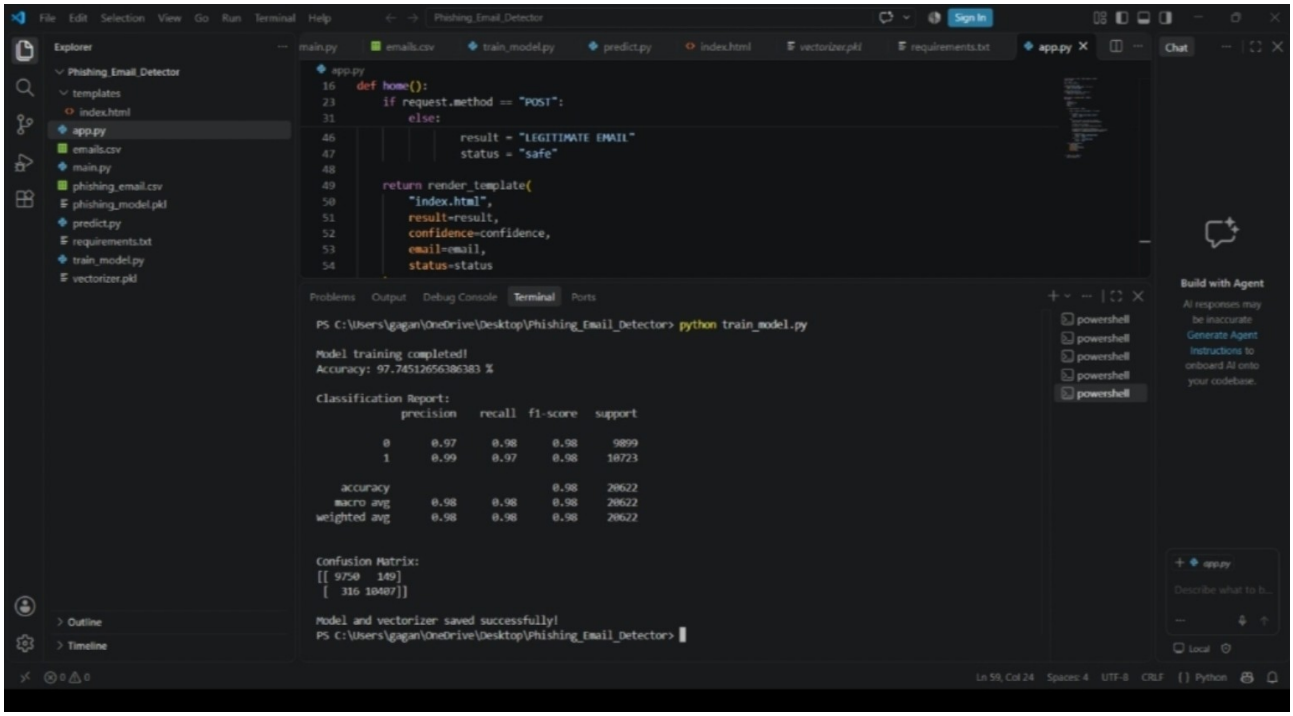
1. Project files in Visual Studio Code

The project folder contains the Flask application (app.py), the HTML interface (templates/index.html), the training and prediction scripts, the CSV datasets, the saved machine-learning model and vectorizer, and the requirements file.



2. Training the machine-learning model

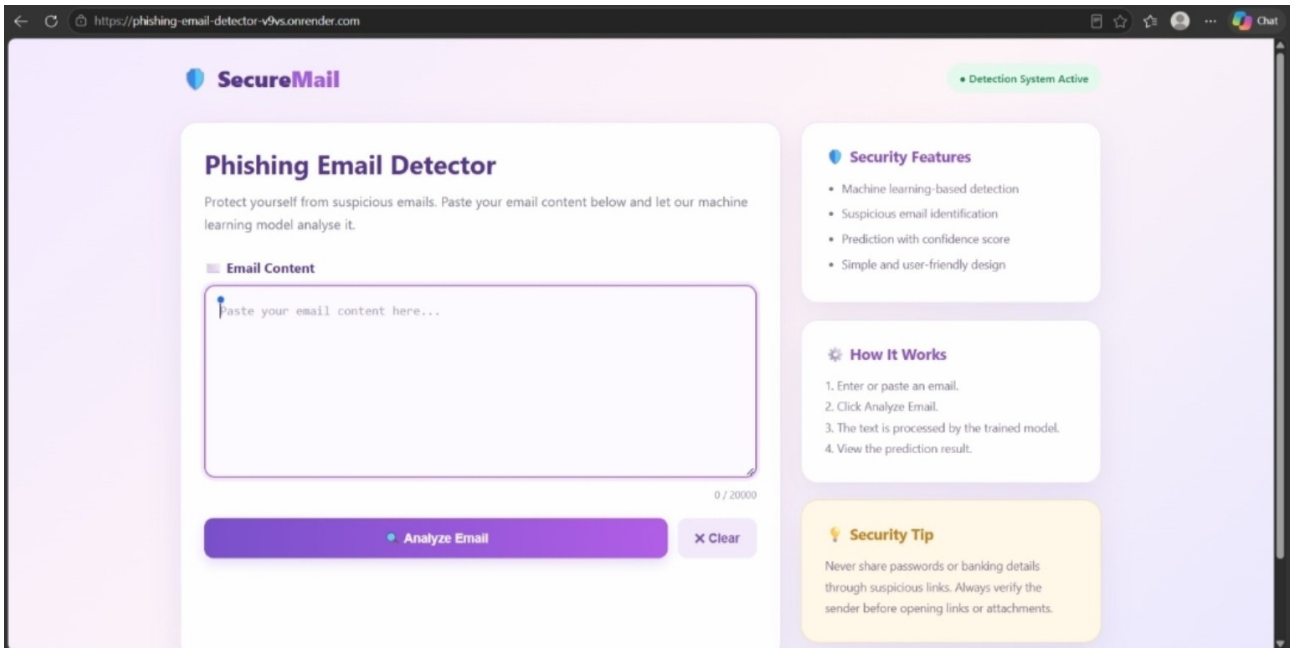
The training script is run with **python train_model.py**. It trains the classifier using the labelled email dataset, reports evaluation metrics, and saves the trained model and text vectorizer for later use. The terminal screenshot shows an accuracy of approximately 97.75%.



The training accuracy shown in the terminal is an evaluation result on the dataset split used by the script. It should not be interpreted as a guarantee that every real-world email will be classified correctly.

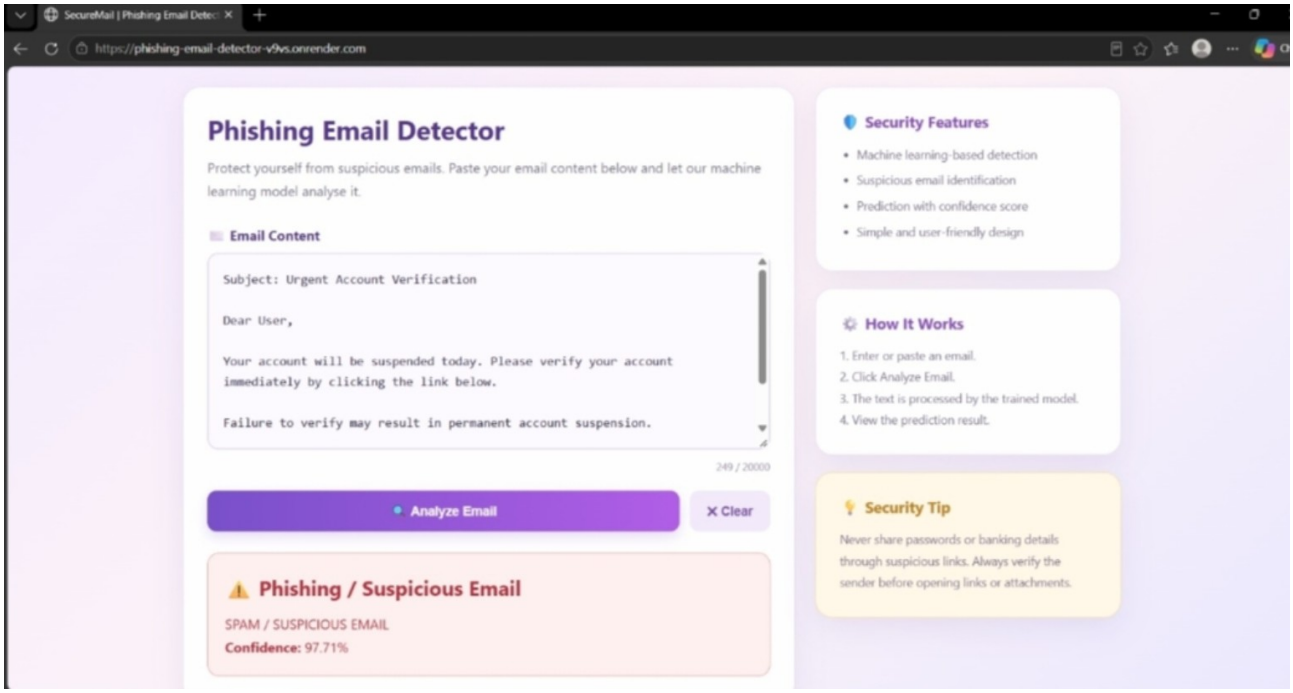
3. Deployed application — home screen

The application is hosted on Render and opens in a browser. The user enters or pastes email text into the Email Content box. The page also explains the security features and the basic steps for using the tool.



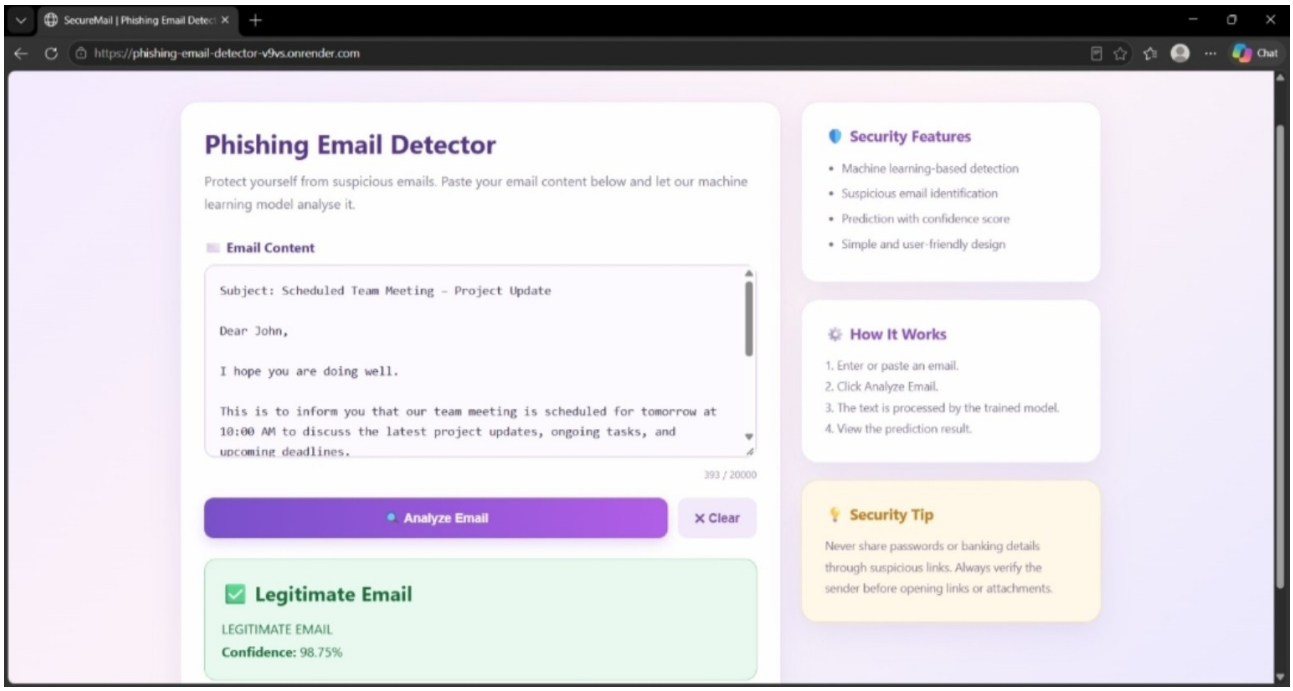
4. Test case — suspicious / phishing email

A sample email uses urgent account-verification language and threatens account suspension. After the user selects Analyze Email, the application displays “Phishing / Suspicious Email” with a confidence score of 97.71% for this example.



5. Test case — legitimate email

A routine team-meeting and project-update email is analysed. The application displays “Legitimate Email” with a confidence score of 98.75% for this example.



Important security note: the prediction is an aid, not a guarantee. Users should still verify the sender and destination of links independently, and should never enter passwords or banking details after following an unexpected email link.